

System Flowcharts

Eight diagrams of how the game works, from one coin to a whole session. Each mirrors the shipped code; the source is named on its page.

01 · Core and meta loops

Serve, pick up, spend; then the next island.

02 · Screen flow

Every screen and how the player moves between them.

03 · A traveller's journey

Arrivals and departures through the counter chain.

04 · The airship timetable

The ship's state machine, with storms and Grand Liners.

05 · Serving and coins

From a queue to a purchase.

06 · What a purchase does

Pad effects, new pads and goals.

07 · Sessions, saving and offline

What happens between sessions.

08 · Code architecture

Rules in pure C#, MonoBehaviours only for display, input and animation.

Colour key

■ Entry / exit point

■ Failure state

■ Special / pure C# rules

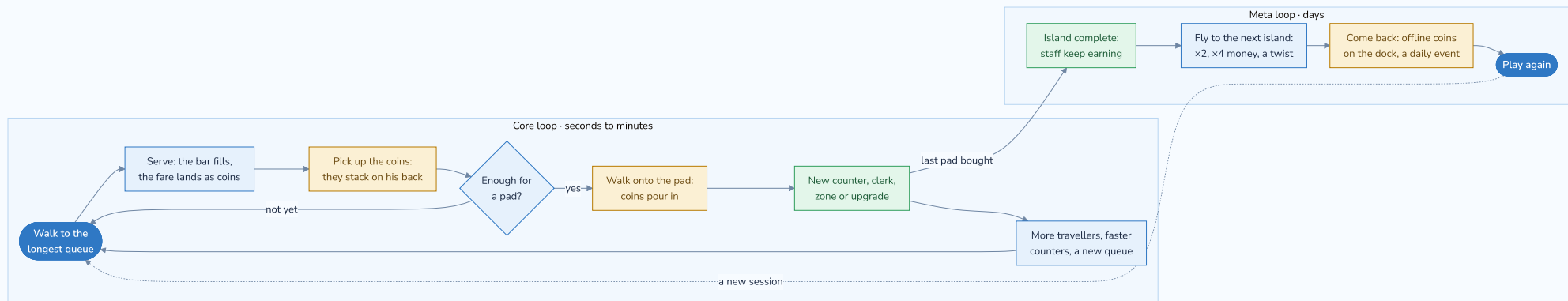
■ Reward or success

■ Coins, ads, services

■ Neutral path

Core and meta loops

Serve, pick up, spend; then the next island.



1 The pad chain guarantees something to buy: the longest wait for a new player is 4:54.

2 Walking is the cost of every action, so where to stand is the decision.

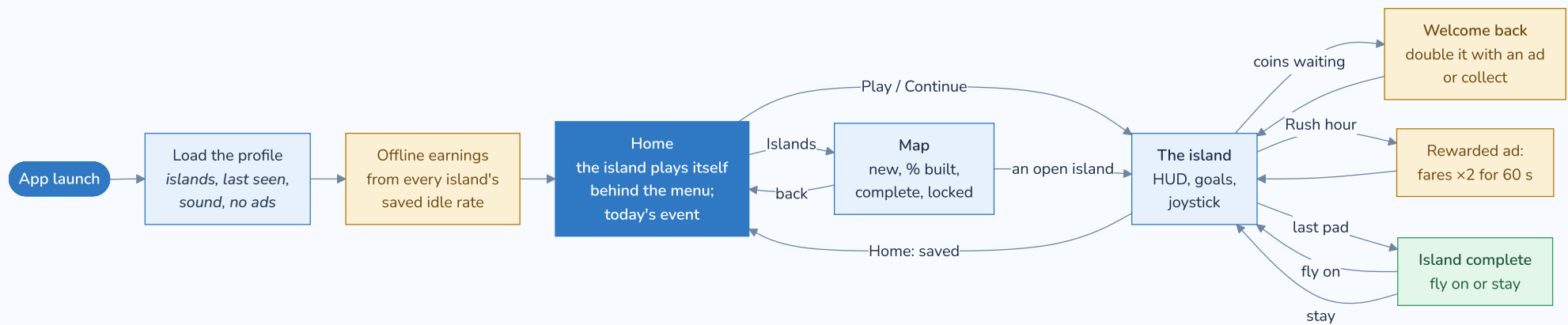
3 Completed islands keep earning, which funds the next one.

- Entry / exit point
- Reward or success
- Failure state
- Coins, ads, services
- Special / pure C# rules
- Neutral path rules

Source: [Island.Tick](#), [Campaign](#)

Screen flow

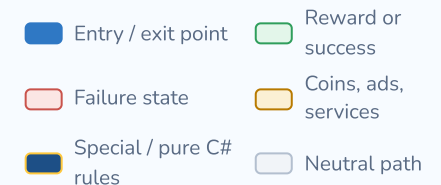
Every screen and how the player moves between them.



1 The home screen runs the current island on the bot, unsaved and silent.

2 Both ads are opt-in and rewarded; no interstitials.

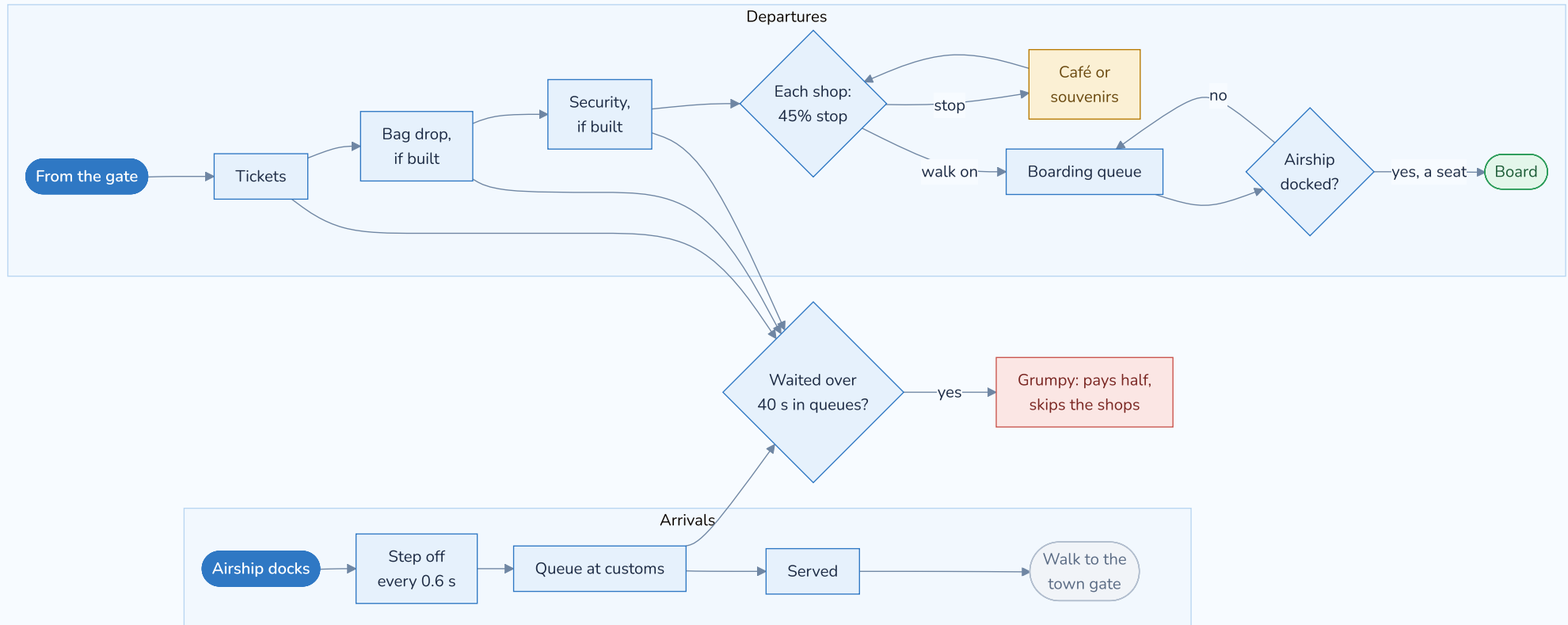
3 Leaving an island always saves it.



Source: [AppController](#)

A traveller's journey

Arrivals and departures through the counter chain.



1 Only built counters are on the route; the chain grows with the island.

2 Each shop tempts a traveller on its own, so the second shop adds customers.

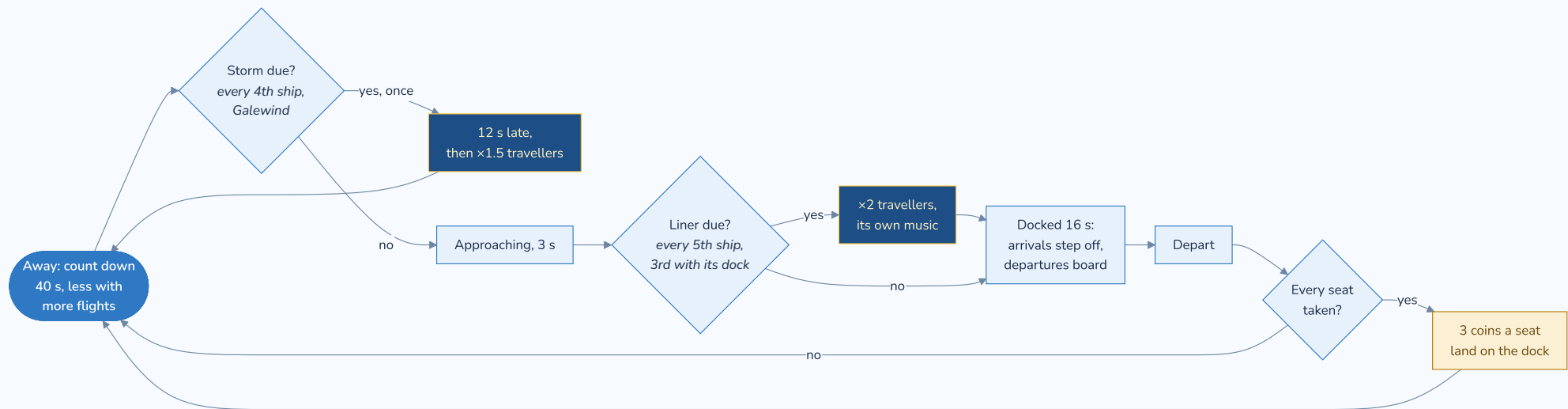
3 Grumpy travellers still pay (half) and still board.

- Entry / exit point
- Reward or success
- Failure state
- Coins, ads, services
- Special / pure C# rules
- Neutral path

Source: [Island.RouteOnwards](#)

The airship timetable

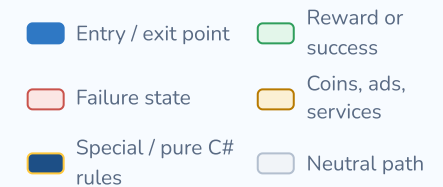
The ship's state machine, with storms and Grand Liners.



1 Supply is capped by the timetable: the pacing lever of the whole island.

2 Storms delay a ship once, then it lands with a bigger crowd.

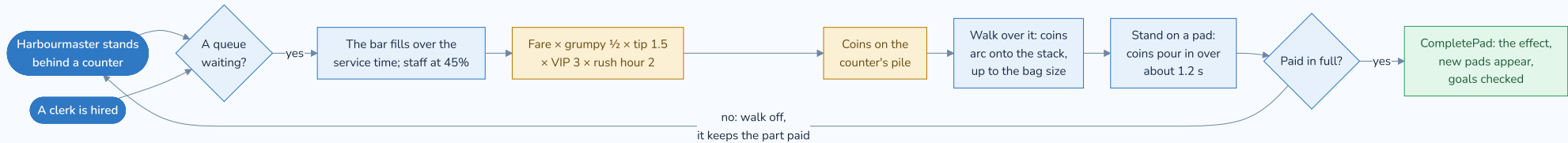
3 The full-ship bonus rewards a moving departure chain.



Source: [Island.TickShip](#), [Depart](#)

Serving and coins

From a queue to a purchase.



1 Staff serve at 45% of the harbourmaster's pace.

2 A traveller served by the harbourmaster tips 50%.

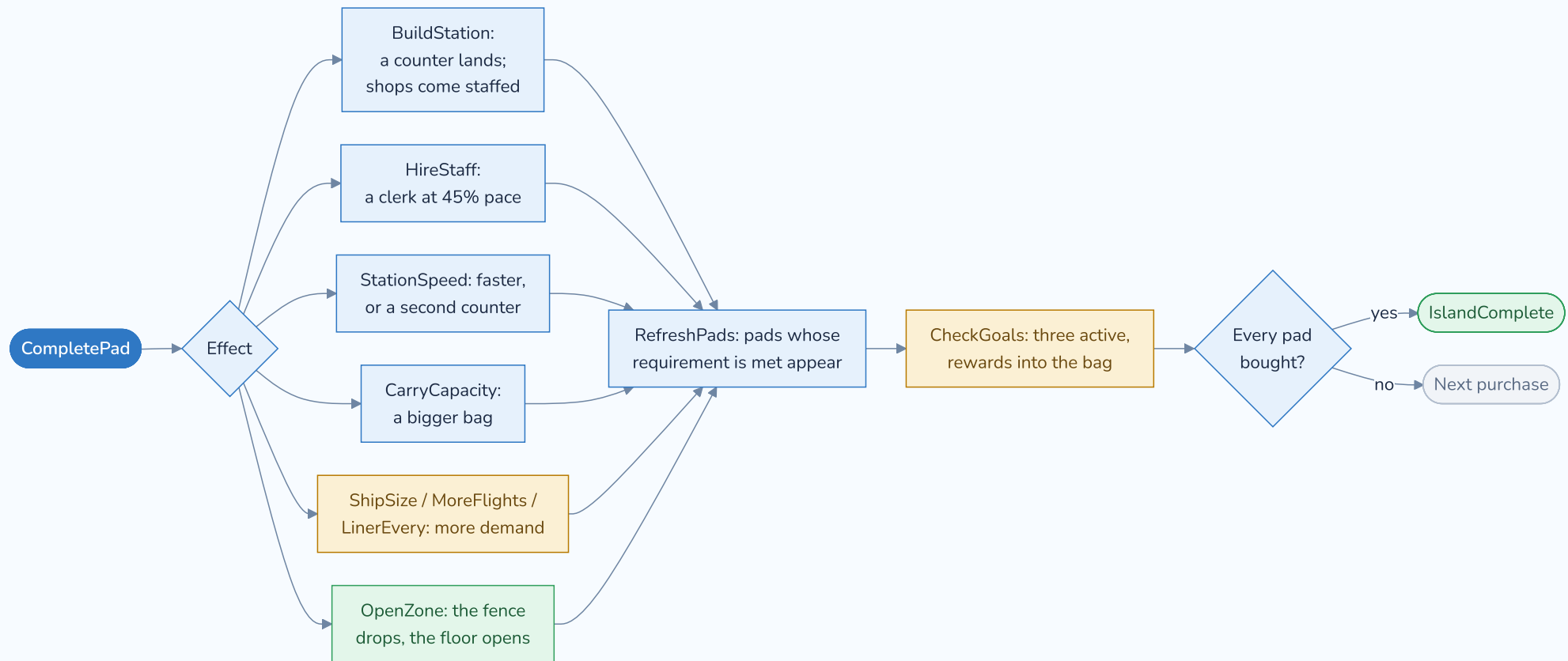
3 A pad keeps what was paid into it.

- Entry / exit point
- Reward or success
- Failure state
- Coins, ads, services
- Special / pure C# rules
- Neutral path

Source: [Island.TickStations, Pay](#)

What a purchase does

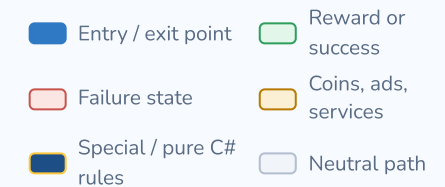
Pad effects, new pads and goals.



1 Each pad appears when the one it requires is bought.

2 Second counters are speed upgrades shown as a booth with a clerk.

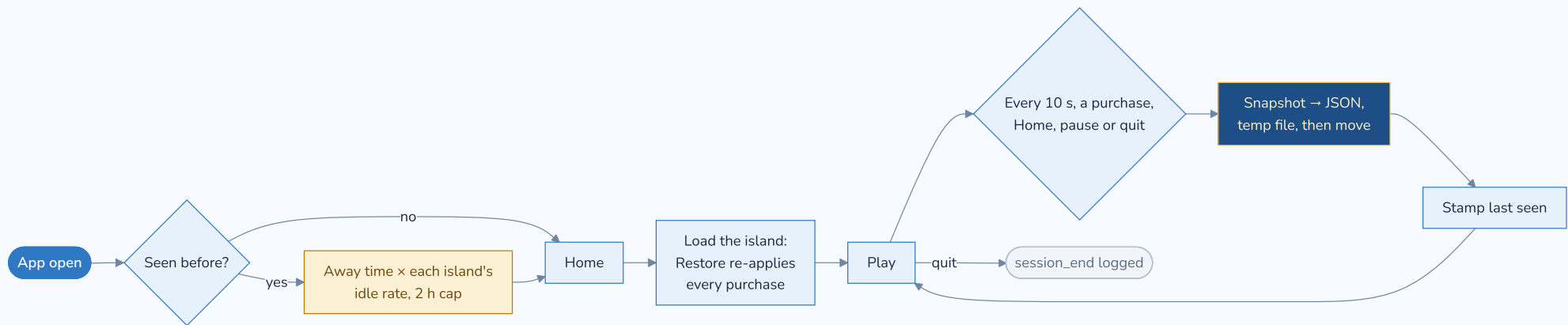
3 Goals pay into the bag, so a reward is spent at once.



Source: [Island.CompletePad](#)

Sessions, saving and offline

What happens between sessions.



1 Idle income is 30% of the played rate: playing always pays more.

2 A broken departure chain earns nothing offline.

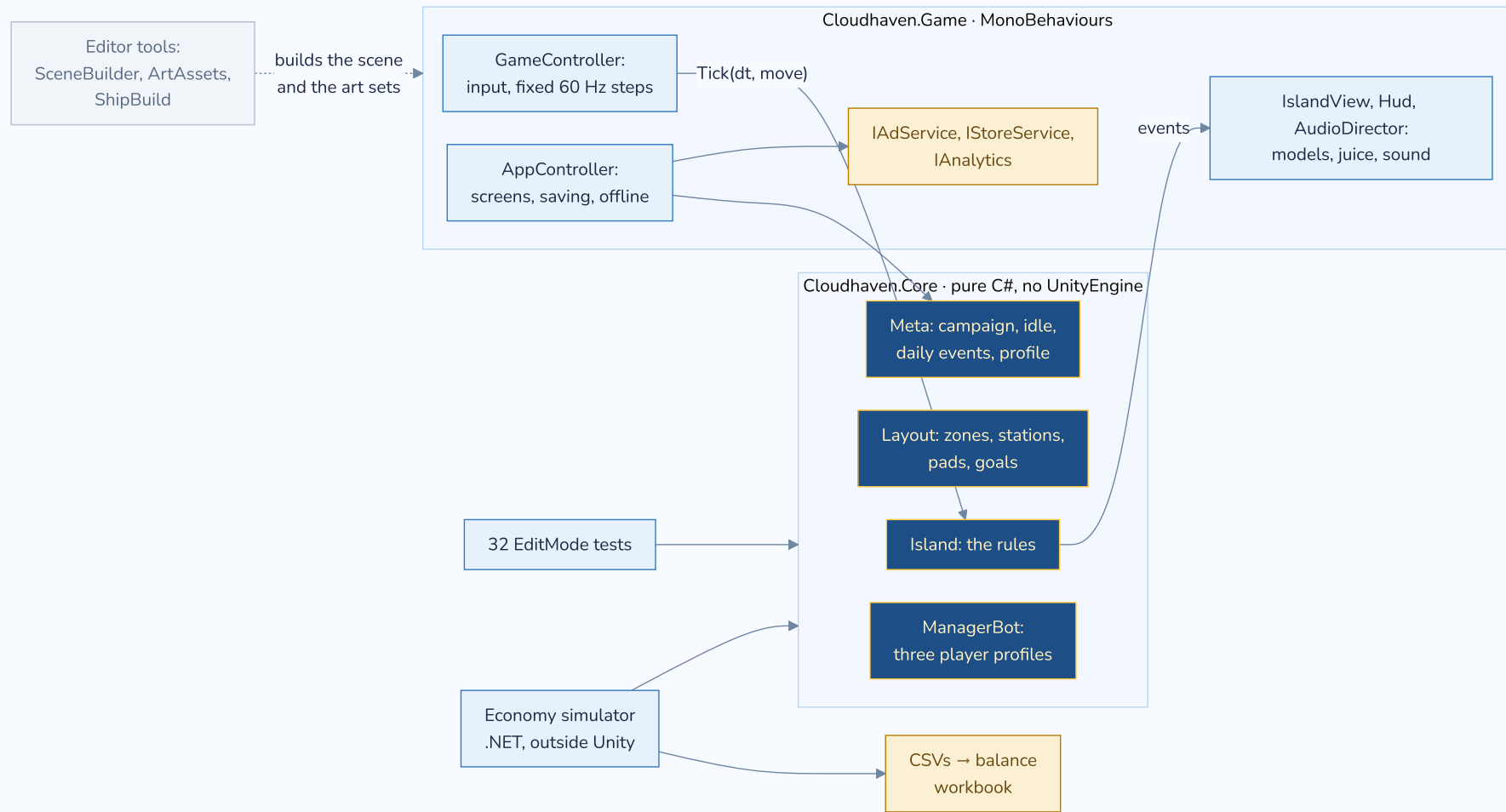
3 Restore uses the same CompletePad as play; a test round-trips an island.

- Entry / exit point
- Reward or success
- Failure state
- Coins, ads, services
- Special / pure C# rules
- Neutral path

Source: `AppController`, `Idle`, `Island.Restore`

Code architecture

Rules in pure C#, MonoBehaviours only for display, input and animation.



1 The same Core runs in the game, the tests and the simulator.

2 The scene and the art sets are built from code, so they can be rebuilt.

3 Colour key: dark = pure C# rules, gold = services and data.

- Entry / exit point
- Reward or success
- Failure state
- Coins, ads, services
- Special / pure C# rules
- Neutral path

Source: [Assets/_Project/Scripts](#)